

Семинар по C++ №15

Шаблоны. Часть 2

Dependent names (зависимые имена)

- В классе может быть объявлен `using`, а может быть статическое поле
- При первой стадии компиляции нужно компилятору сказать, как парсить
- По умолчанию считается полем
- Можно явно сказать, что это название типа: `typename`

Dependent names (using)

```
template <typename T>
void f() {
    S<T>::A* x; // это статическое поле с умножением или такой тип
    объявлен в классе?
    // Считаю по умолчанию, что это поле
}
```

- template <typename T>
- void f() {
- **typename S<T>::A*** x; // это тип
- }

Dependent names (template method)

- В классе может быть объявлена шаблонная функция, а может быть статическое поле
- При первой стадии компиляции нужно компилятору сказать, как парсить
- По умолчанию считается полем
- Можно явно сказать, что это шаблон функции: `template`

Dependent names (template method)

```
template <typename T>
void f() {
    S<T> s;
    S.foo<5>(x + y); // это статическое поле и сравнения или такой шаблон функции объявлен в классе?
    // Считаю по умолчанию, что это поле
}
```

```
template <typename T>
void f() {
    S<T> s;
    S.template foo<5>(x + y); // это шаблон функции
}
```

Dependent names (template base class)

- При обращении к полям шаблонного родителя из наследника компилятор не находит поля (потому что иначе пришлось бы проверять всех шаблонных родителей)
- Нужно явно указать, что это поле через `this` или `Base<T>::field_name`

Dependent names (template base class)

```
template <typename T>
struct Base {
    int x;
};
```

```
template <typename T>
struct Derived : Base<T> {
    int f() {
        return x; // CE
        return Base<T>::x;
        return this->x;
    }
};
```

Type traits

- Метафункции – работаем с типами вместо объектов
- Описаны в `type_traits`
- Базовый пример:

```
template <typename T, typename U>
struct is_same {
    static constexpr bool value = false;
};
```

```
template <typename T>
struct is_same<T, T> {
    static constexpr bool value = true;
};
```

```
template <typename T, typename U>
bool is_same_v = is_same<T, U>::value;
```

Type traits

- Вместо constexpr bool value = false / true можно наследоваться от std::false_type или std::true_type
- Можно использовать шаблонные переменные / using с помощью *_v, *_t
- std::conditional<bool, type1, type2> - мета тернарный оператор
Если условие true, conditional_t равен type1, иначе type2
- Некоторые (простые) [type traits](#):
 - is_same, remove_const, remove_reference, conditional, decay, is_member_pointer, rank

Variadic templates

- Шаблоны с переменным числом аргументов
- Частый паттерн: разделяем шаблонные параметры на голову и остальные параметры, внутри вызываем от хвоста
- Определяем базу
- Оператор sizeof... - возвращает количество объектов в пакете

```
template <typename Head, typename... Tail>
void print(const Head& head, const Tail&... tail) {
    std::cout << head;
    print(tail...);
}
```

Fold expressions

- [Свертка](#) пакета
- Обязательно в скобках (иначе не парсится)
- Пример:

Syntax

(<i>pack op ...</i>)	(1)
------------------------	-----

(<i>... op pack</i>)	(2)
------------------------	-----

(<i>pack op ... op init</i>)	(3)
--------------------------------	-----

(<i>init op ... op pack</i>)	(4)
--------------------------------	-----

1) Unary right fold.

2) Unary left fold.

3) Binary right fold.

4) Binary left fold.

```
template <typename Head, typename... Tail>
struct is_homogeneous {
    static constexpr bool value = (std::is_same_v<Head, Tail> && ...);
};
```

C RTP (curiously recurring template pattern)

- Хотим задать интерфейс
- Наследник наследуется от родителя с шаблонным параметром равным самому себе
- Для того, чтобы достать указатель на implementation, можно скастить себя (this) вниз к шаблонному параметру (так как он является наследником)
- В классе родителя задаём интерфейс, методы вызываются от implementation

C RTP (example)

```
template <typename Implementation>
struct ParserInterface {

    int getData() {
        return impl()->getDataImpl();
    }

    int getID() {
        return impl()->getIDImpl();
    }

    Implementation* impl() {
        return static_cast<Implementation*>(this);
    }
};
```

```
struct ParserJsonImpl : public
ParserInterface<ParserJsonImpl> {

    int getDataImpl() {
        ...
    }
    int getIDImpl() {
        ...
    }
};
```